

Test Automation

Backend

1st edition

Module 2: Request Types

Bruno Machado

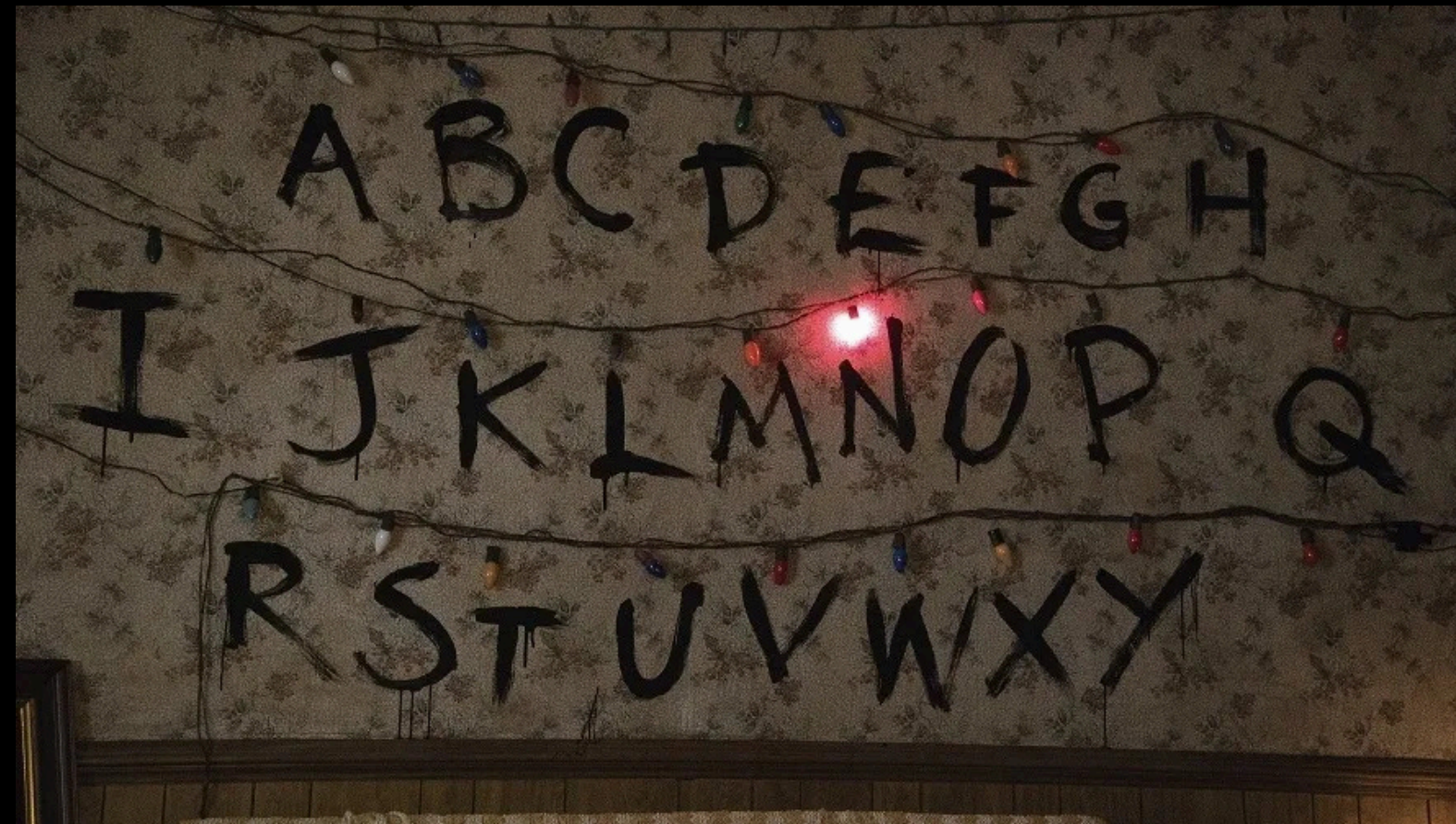
HTTP Methods

HTTP Methods help communicate which **action** you want to perform with the data on the web.

GET	POST
PUT	DELETE
PATCH	HEAD
OPTIONS	TRACE
CONNECT	

GET

GET is used to **retrieve or view data** that is already available, such as viewing a product page, a list of movies, or user details, without modifying or adding anything.



GET

ENDPOINT

GET /api/characters/will-byers

RESPONSE

```
{  
  "name": "Will Byers",  
  "status": "Missing",  
  "location": "Upside Down",  
  "notes": "Trapped in the Upside Down, unable to communicate"  
}
```


POST

POST is used to **send data** to the server, creating or updating a resource. For example, it can be used to add a new product, register a user, or send a message, modifying or adding information on the server.



POST

ENDPOINT

POST /api/characters/hopper/teleport

REQUEST BODY

```
{  
  "name": "Jim Hopper",  
  "role": "Sheriff",  
  "location": "Hawkins",  
  "skills": ["combat training", "investigation", "leadership"],  
  "status": "active"  
}
```

RESPONSE BODY

```
{  
  "message": "Jim Hopper successfully sent to the Upside Down",  
  "status": "success",  
  "character": {  
    "name": "Jim Hopper",  
    "role": "Sheriff",  
    "status": "active",  
    "destination": "Upside Down",  
    "mission": "Rescue"  
  }  
}
```

PUT

PUT is used to **update** data on the server, completely replacing an existing resource or creating a new one if it doesn't already exist.



PUT

ENDPOINT

PUT /api/characters/will-byers

REQUEST BODY

```
{  
  "name": "Will Byers",  
  "status": "Recovered",  
  "location": "Hawkins",  
  "notes": "Experiencing Upside Down visions, but physically safe"  
}
```

RESPONSE BODY

```
{  
  "message": "Will Byers' status successfully updated",  
  "status": "success",  
  "character": {  
    "name": "Will Byers",  
    "status": "Recovered",  
    "location": "Hawkins",  
    "notes": "Experiencing Upside Down visions, but physically safe"  
  }  
}
```


DELETE

DELETE is used to **remove** a resource from the server.

When using **DELETE**, you are telling the server that a specific resource (such as a character, an item, or an entity) should be **permanently deleted**.



DELETE

ENDPOINT

DELETE /api/creatures/demogorgon

RESPONSE BODY

```
{  
  "message": "Demogorgon has been successfully deleted",  
  "status": "success",  
  "creature": {  
    "name": "Demogorgon",  
    "danger_level": "extreme",  
    "origin_location": "Upside Down",  
    "status": "removed"  
  }  
}
```

PATCH

PATCH is used to partially update existing data, meaning you **can modify only specific fields** of a resource without replacing the entire object.

It is commonly used when you want to update just a few details, such as changing a user's email or updating a character's level, without affecting the rest of the data.

PATCH

ENDPOINT

PATCH /api/characters/will-byers

REQUEST

```
{  
  "status": "Found"  
}
```

RESPONSE

```
{  
  "message": "Character updated successfully",  
  "status": "success",  
  "character": {  
    "name": "Will Byers",  
    "status": "Found",  
    "location": "Hawkins",  
    "notes": "Recovered from the Upside Down"  
  }  
}
```

STATUS CODE

Status codes are numbers returned by the server to indicate the result of a request. They help communicate whether the operation was successful or if there was a problem.

1xx	INFORMATIVE	Request received and being processed.	100 Continue
2xx	SUCCESS	Request successfull	200 OK, 201 Created
3xx	REDIRECT	Request needs more actions to be continued.	301 Moved Permanently
4xx	CLIENT ERROR	Invalid request made by the client.	400 Bad Request, 404 Not Found
5xx	SERVER ERROR	Server failed to process this request.	500 Internal Server Error